Course Project on

ECE 207 – Control Systems

AY 2025-2026 (Semester 4)

Submitted as a part of Continuous Learning Assessment – 3

**Root Locus Based Speed Control of DC Motor using MATLAB/Simulink**


Department of Electronics and Communication Engineering,

School of Engineering and Sciences,

SRM University-AP, Andhra Pradesh


Details of Project Group

| Section | ECE-C | |
|---|---|---|
| Group No | C2-7 | |
| Member 1 | AP24110020159 | U Sai Hanvitha |
| Member 2 | AP24110020166 | N Aunnitya Gandhi |
| Member 3 | AP24110020168 | M Taraka Ram |

Signature of course instructor

(Dr. Anuj Deshpande)

Individual Contribution

Clearly mention contribution to the project, without any overlap with other members.

| Sr. No. | Registration No. | Contribution to the project |
|---|---|---|
| 1 | AP24110020166 | Worked on building the Simulink model setting transfer function parameters from MATLAB |
| 2 | AP24110020159 | Worked on DC motor mathematical modeling |
| 3 | AP24110020168 | Worked on explaining control concepts and analyzing simulation results |

# Brief of the project

This time, using control theory and MATLAB/Simulink, we will attempt to gain precise and smooth control over the speed of the DC motor – an electric motor that can be used to build and model other real systems (fans, robots, conveyors belts, etc.). In each of these systems, we do not just want the motor to rotate, but we want it to be able to rotate to a specific speed and maintain that speed regardless of external disruptions. We can begin here by modelling the DC motor as a "system". Within systems theory, the system itself has an input and an output. Based upon this concept, the input to such a DC motor system may be the applied armature voltage, while the output of the system is the resulting angular speed of the motor. Using these electrical and mechanical equations of the motor, we can derive a transfer function – a model of the system that characterizes the relationship between input and output of that system (the relationship between speed and voltage). This transfer function will allow for the simulation of the motor and an examination of the inherent response of the motor to an applied input voltage. In this case of the input voltage, the motor response will likely not be optimal. Indeed, the motor may take an unreasonable amount of time to reach a steady state, or it may fail altogether to reach the desired speed. Thus, we start first with the utilization of root locus techniques – this allows us to see how the closed-loop poles of the system move with change in controller gain. Such poles can inherently provide fast, stable and minimal overshoot response when drawing root locus plots using MATLAB. Start off with a proportional (P) controller then eventually transition to using a PI controller as the integral action would help the motor correct for steady-state errors and reach your desired speed to a greater accuracy.

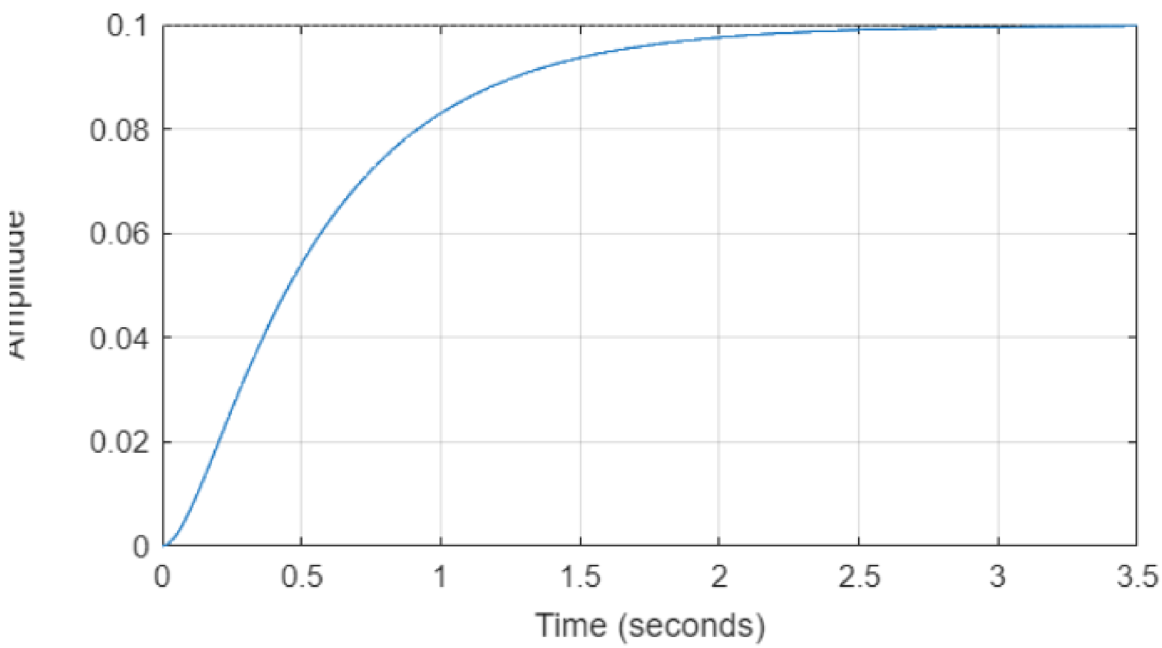
Simulink will be where all this magic happens. We will build a block diagram resembling that of an actual control system. A step block for your desired speed. A sum block for your error. A transfer function block for your controller. Another transfer function block for your motor and feedback to supply your actual speed to compare back to. We will be able to simulate our system and see a speed vs time plot. Watch how an uncompensated motor performs, then see how it performs when using a P controller, and finally a PI controller.

Building a DC motor, Simulating it with mathematics, Using root locus to design a controller, and finally implementing it into real life with Simulink.

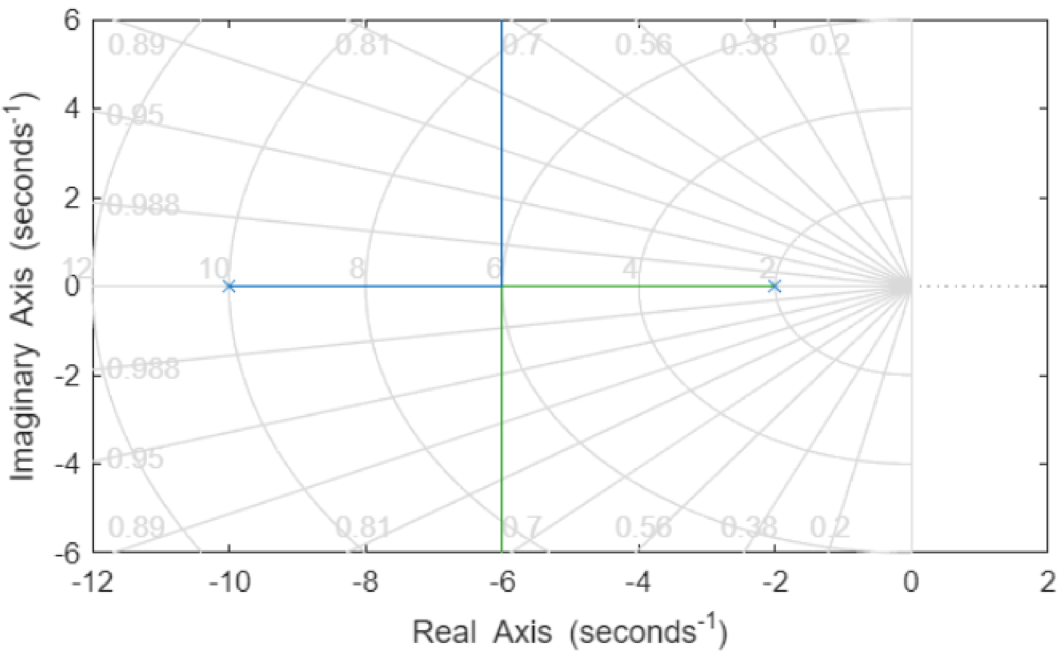
## Results:

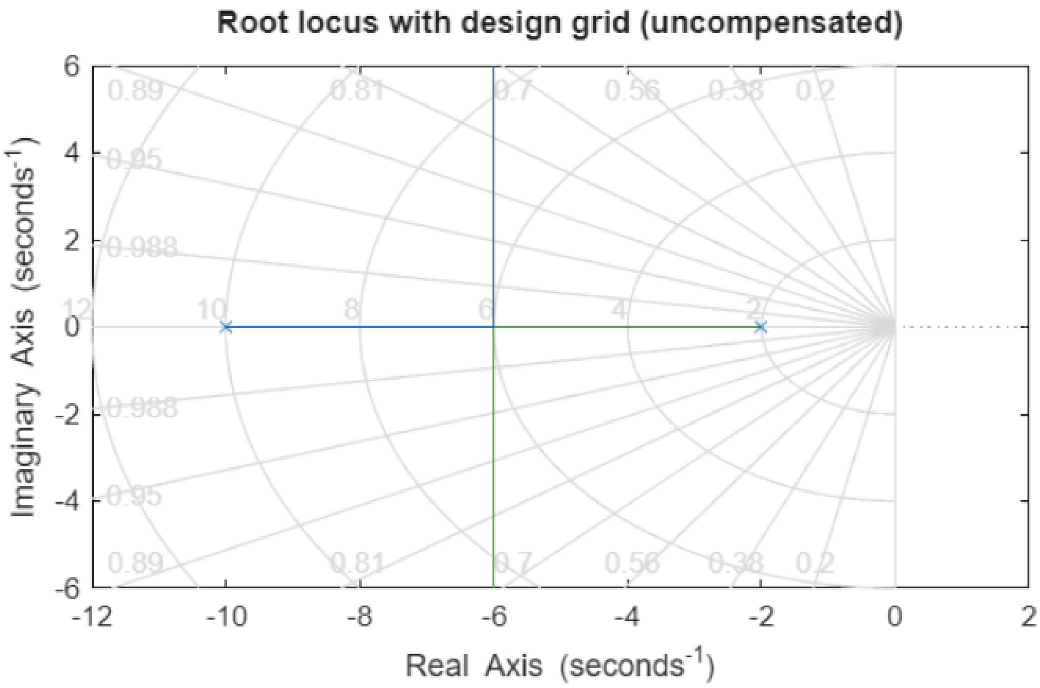
Simulation results demonstrate improved DC motor speed response using root-locus-based controller design versus the uncompensated system. The uncompensated system (the open-loop motor and simple unity-feedback case) shows slower settling and visible steady-state error to a step speed command. When a proportional controller tuned using the root locus is added, the motor reaches commanded speed quicker and with less steady-state error. However, there is still some steady-state error and overshoot present. By adding a PI controller with gain and zero selected using the compensated root locus, we can make the system have a much smaller steady-state error, while keeping settling time and overshoot reasonable

**Open-loop step response of DC motor speed**

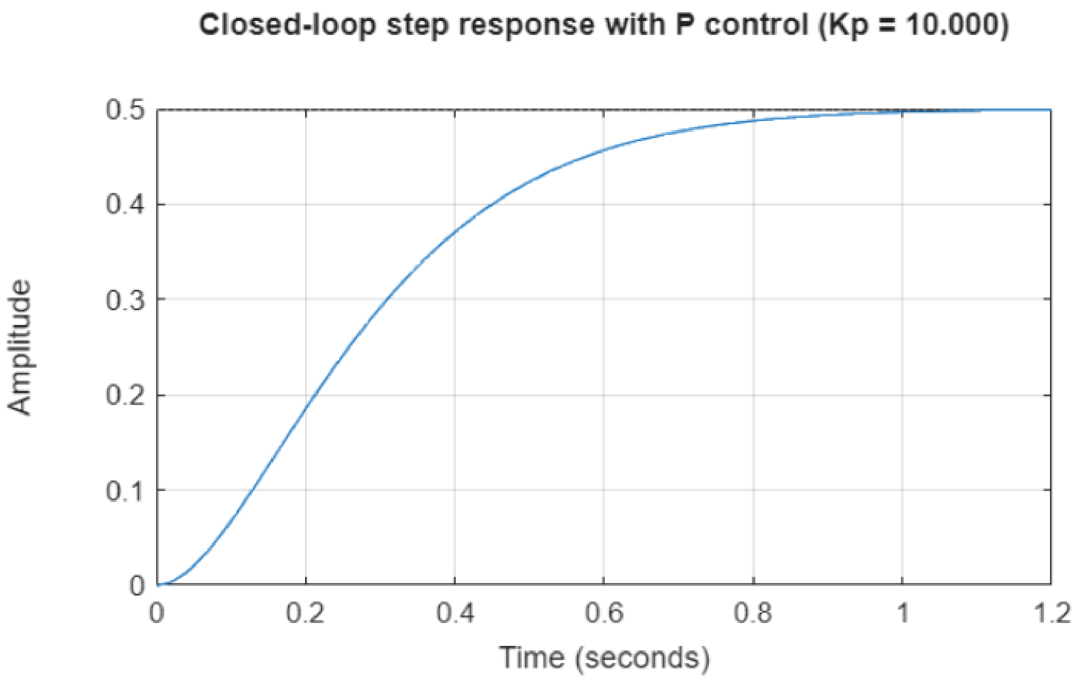



**Root locus of uncompensated DC motor plant**





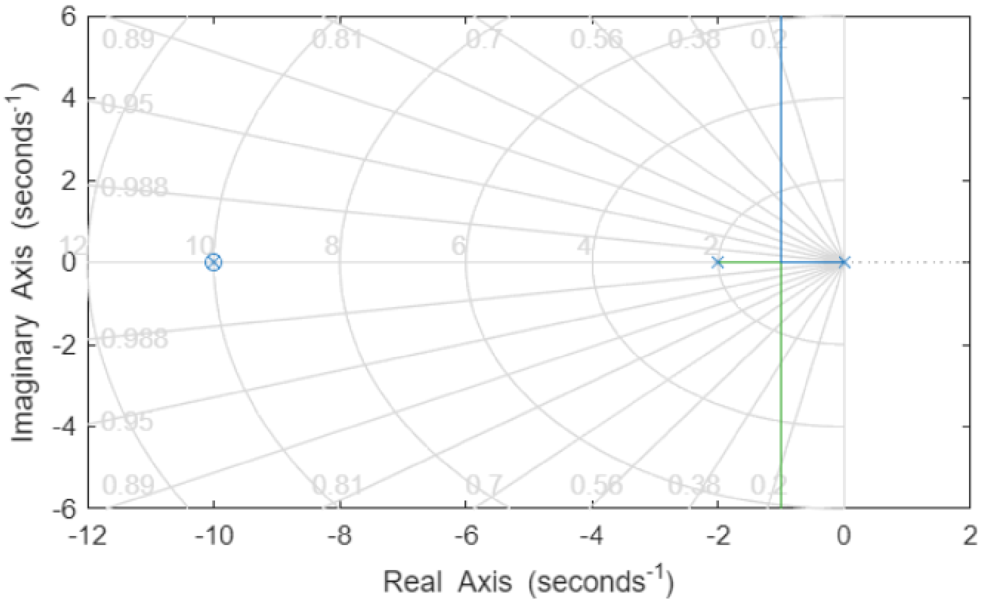

hosen proportional gain Kp = 10



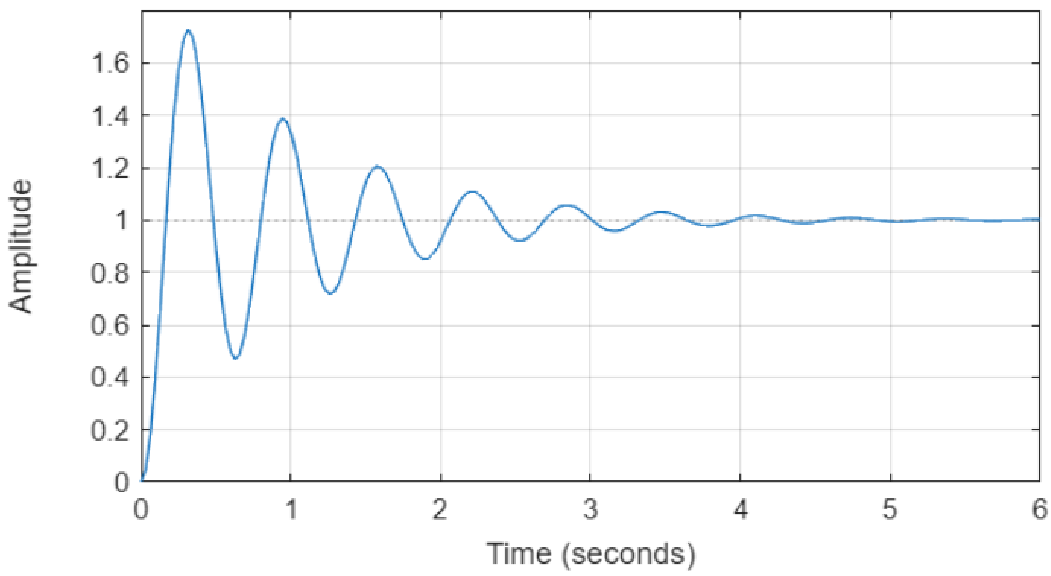

Chosen PI zero zc = 10

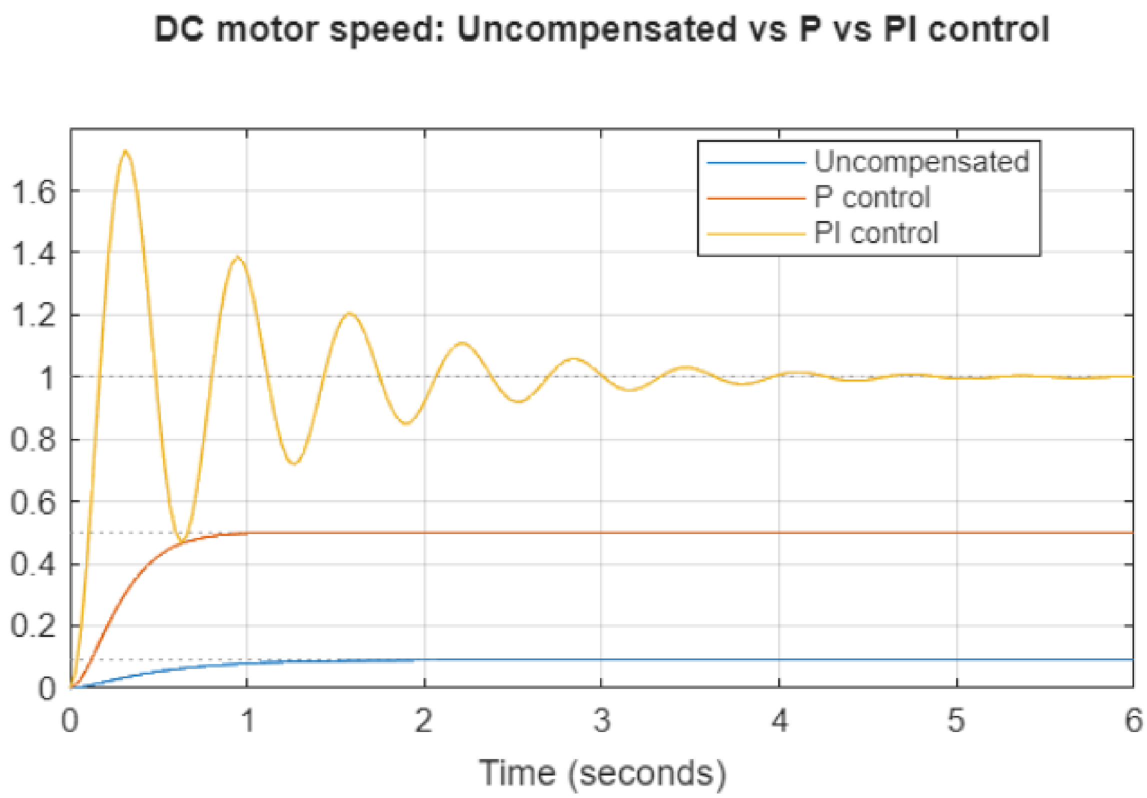
Chosen PI proportional gain Kp_pi = 50

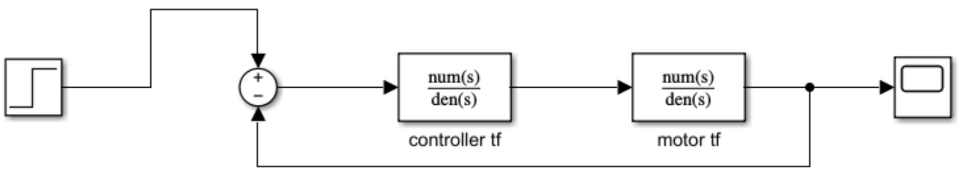
**Root locus of PI-compensated plant (structure only)**

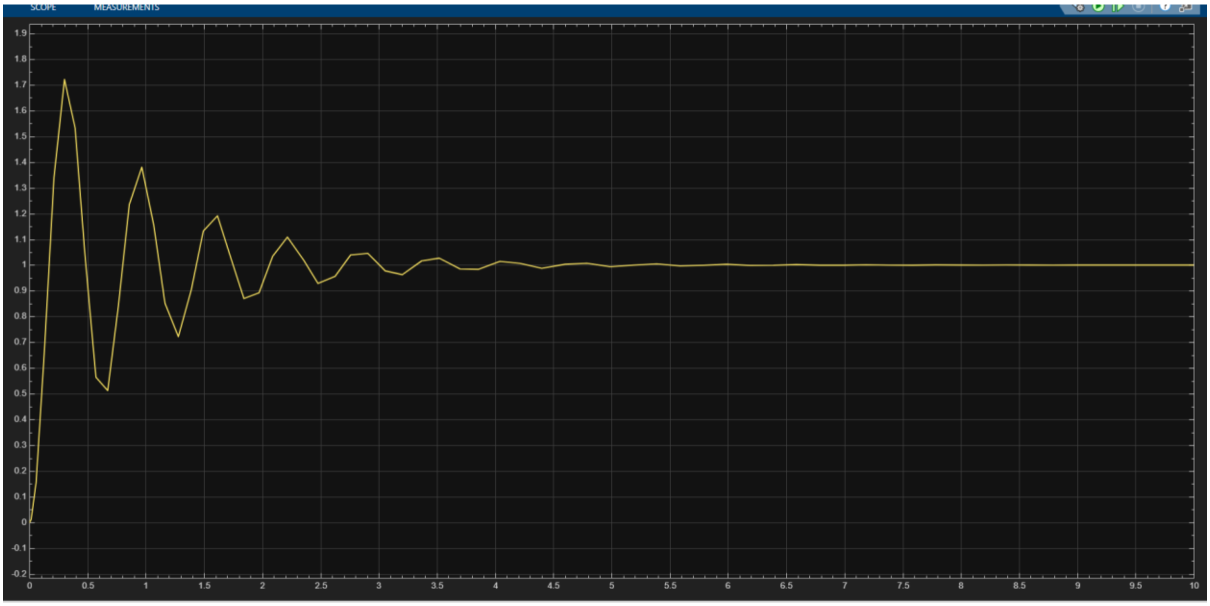



**Closed-loop step response with PI control (Kp_pi = 50.000, zc = 10.0)**




DC motor speed: Uncompensated vs P vs PI control
Uncompensated
P control
PI control
1.6
1.4
1.2
1
0.8
0.6
0.4
0.2
0
0
1
2
3
4
5
6
Time (seconds)


+
−
num(s)
den(s)
controller tf
num(s)
den(s)
motor tf


SCOPE
MEASUREMENTS
1.9
1.8
1.7
1.6
1.5
1.4
1.3
1.2
1.1
1
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
-0.1
-0.2
0 0.5 1 1.5 2 2.5 3 3.5 4 4.5 5 5.5 6 6.5 7 7.5 8 8.5 9 9.5 10